Project Executable Files

Date	03 October 2026
Team ID	TEAM-ID-TO-BE-FILLED
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes (`README.md`)
3	requirements.txt / package.json / dependency file	Yes (`requirements.txt`, `requirements-dev.txt`)
4	Database schema / seed files (if applicable)	Yes (schema in `database.py`, created on start; `tools/seed_demo_user.py`)
5	Environment configuration file (.env.example or similar)	Yes (`.env.example`)
6	Deployed application URL (if hosted)	NA (runs locally)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (web application)
8	Dockerfile / Containerization config (if applicable)	Yes (`Dockerfile`, `.dockerignore`)
9	Test files and test results	Yes (`tests/`, `6.Project Testing/evidence/`)
10	Demo video or walkthrough (if required)	To be added after recording

Step 2: File / Folder Structure

```
Pocketsmart-AI/
|-- main.py                    FastAPI app: routes, sessions, wiring
|-- gemini_utils.py            Gemini service: prompts, budget validation, shopping links, fallback
|-- auth.py                    Password hashing, JWT, token blacklist, sessions
|-- models.py                  Pydantic input / session schemas
|-- database.py                SQLite schema and queries
|-- config.py                  Settings from environment variables
|-- requirements.txt           Runtime dependencies
|-- requirements-dev.txt       Test and documentation tooling
|-- .env.example               Environment template (copy to .env)
|-- Dockerfile, .dockerignore  Container build
|-- templates/                 base, index, login, register, dashboard, home_planner,
|                              party_planner, jewelry_planner, history (.html)
|-- static/                    styles.css, app.js, uploads/
|-- tests/                     test_app.py (43 tests), conftest.py
|-- tools/                     check_gemini.py (Activity 1.3 connectivity check), perf_load.py (loa
|                              seed_demo_user.py, capture_screenshots.py, build_docs.py
|-- docs-src/                  Editable Markdown sources and project.json for the PDFs
|-- 1. Brainstorming & Ideation/ ... 8.Project Demonstration/   Phase deliverables (PDF)
`-- README.md
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not hosted - runs locally at http://localhost:8000 (see run instructions)
Login Credentials (Demo)	Username `demo`, password `Demo@12345` (created by `python tools/seed_demo_user.py`), or register a new account
Platform / Hosting Provider	Local Uvicorn server; Docker image available for Render, Railway, AWS or Azure
Repository Link	https://github.com/kaushalravin/Pocketsmart-AI
Demo Video Link	To be added after recording

Step 4: Run Instructions

1. Install Python 3.10 or newer and Git.
2. `git clone https://github.com/kaushalravin/Pocketsmart-AI` then `cd Pocketsmart-AI`
3. Create and activate a virtual environment: `python -m venv venv` then `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (macOS/Linux).
4. `pip install -r requirements.txt`
5. Copy `.env.example` to `.env`. Put your Gemini key from https://aistudio.google.com/apikey in `GOOGLE_API_KEY` and set `SECRET_KEY` to a long random string. Without a key the app still works in offline-fallback mode.
6. Optional checks: `python tools/check_gemini.py` (confirms Gemini access) and `python tools/seed_demo_user.py` (creates the demo account).
7. Start the server: `uvicorn main:app --reload` (or `python main.py`).
8. Open http://localhost:8000, sign in, and use the Home, Party and Jewelry planners.
9. Run the tests: `pip install -r requirements-dev.txt` then `pytest tests`.
10. Docker alternative: `docker build -t pocketsmart-ai .` then `docker run -p 8000:8000 --env-file .env pocketsmart-ai`.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Shopping links are search URLs, not live product listings or prices	Use the links to confirm price; retailer APIs are planned
2	Prices are AI estimates and may differ from current store prices	Plans show estimates; the budget check uses the estimates
3	Sessions and the token blacklist are in memory and reset on server restart	Users sign in again; Redis planned for production
4	Gemini model names change over time (the brief's "1.5 Flash Pro" is retired)	Set `GEMINI_MODEL` in `.env`; the app also tries fallback models automatically
5	Without a valid API key the app returns standard offline estimates, and outfit photos are not analysed	Add `GOOGLE_API_KEY`; the result page labels offline plans